src/vectorstore.py

The FAISS vector store — the one place the corpus is searched

This module builds, saves, loads, and searches the **FAISS** index. It is the single definition of retrieval in the project: both the naive and agentic paths call **VectorStore.search**, so a measured difference between them is always about strategy, never an accidental difference in search.

Walkthrough of all 116 lines, in order. Left numbers are the real line numbers in the file.

```
 1  """The FAISS vector store — build, save, load, and search the index.
 2
 3  This is the single definition of "search the corpus" in the whole project. Both
 4  the plain RAG pipeline (`rag.py`) and the agentic system (`agent.py`) call
 5  `VectorStore.search`; neither re-implements retrieval. That is deliberate — if
 6  there were two search paths they could disagree, and no published number could be
 7  trusted. (Same "single source of truth" rule the rest of the portfolio applies to
 8  its core logic.)
 9
10  Why FAISS? It is an in-process, file-backed nearest-neighbour index — no server,
11  no Docker, no network. The whole thing runs on a laptop, which is what keeps the
12  retrieval path reproducible top-to-bottom in the capstone notebook.
13
14  Why an inner-product index? Our embeddings are L2-normalized (see embeddings.py),
15  so inner product == cosine similarity. `IndexFlatIP` does an *exact* search — it
16  compares the query against every chunk. For a few thousand chunks that is
17  instant, and it removes approximate-search recall as a variable, so retrieval
18  quality reflects the embeddings and chunking, not an ANN approximation.
19  """
```

Lines 1 to 19 are the module docstring. They make three arguments: this is the single source of truth for search (both pipelines call it, so they cannot disagree); FAISS is chosen because it is in-process and file-backed with no server or Docker; and an inner-product index is used because the embeddings are normalized, so inner product equals cosine similarity, and the exact IndexFlatIP removes approximate-search recall as a confounding variable.

```
20
21  from __future__ import annotations
22
23  from dataclasses import dataclass
24
25  import numpy as np
26  import pandas as pd
27
28  from . import config
29  from .embeddings import embed_query, embed_texts
```

Lines 20 to 29 are imports. Line 21 enables postponed annotations. Line 23 imports dataclass for the Retrieved type. Lines 25 to 26 import numpy and pandas (arrays and the metadata frame). Line 28 imports config; line 29 imports the two embedding helpers, embed_query and embed_texts, that turn text

into the vectors FAISS stores and searches.

```
30
31
32  @dataclass
33  class Retrieved:
34      """One search hit: a chunk, its citation metadata, and its similarity score."""
35
36      chunk_id: str
37      arxiv_id: str
38      title: str
39      text: str
40      score: float        # cosine similarity in [-1, 1]; higher is more similar
```

Lines 30 to 40 define the Retrieved dataclass — one search hit. Its fields are the chunk_id (line 36), the arxiv_id and title used for citation (lines 37 to 38), the chunk text itself (line 39), and the similarity score (line 40, cosine in the range minus one to one, higher meaning more similar). The comment on line 40 documents that range.

```
41
42
43  class VectorStore:
44      """An in-memory FAISS index plus the row-aligned chunk metadata.
45
46      The index stores only vectors; the metadata frame stores the text and
47      citation fields. They are kept in the same row order, so FAISS returning row
48      `i` means "chunk at metadata row `i`". Keeping them aligned is the one
49      invariant this class protects.
50      """
51
52      def __init__(self, index, meta: pd.DataFrame):
53          self._index = index
54          self._meta = meta.reset_index(drop=True)
```

Lines 41 to 54 open the VectorStore class and its constructor. The docstring (lines 44 to 50) states the one invariant the class protects: the FAISS index and the metadata frame are kept in the same row order, so FAISS returning row i means the chunk at metadata row i. The __init__ (lines 52 to 54) stores the index and calls reset_index(drop=True) on the metadata so its row labels are a clean 0..n range that lines up with FAISS row numbers.

```
55
56      # ---- construction ----------------------------------------------------- #
57      @classmethod
58      def build(cls, chunks: pd.DataFrame) -> "VectorStore":
59          """Embed every chunk and build the index. This is the expensive step."""
60          import faiss
61
62          vectors = embed_texts(chunks["text"].tolist())
63          index = faiss.IndexFlatIP(config.EMBED_DIM)
64          index.add(vectors)
65          # Store exactly the columns needed to return and cite a hit.
66          meta = chunks[["chunk_id", "arxiv_id", "title", "text"]].copy()
67          return cls(index, meta)
```

Lines 55 to 67 define build(), a classmethod that constructs a store from a chunk frame — the expensive step. faiss is imported inside the method (line 60) so importing the module stays light. Line 62 embeds every chunk's text. Line 63 creates a flat inner-product index of the right dimension. Line 64 adds the vectors. Line 66 keeps only the four columns needed to return and cite a hit. Line 67 returns a new instance via cls(index, meta).

```
68
69      # ---- persistence ------------------------------------------------------ #
70      def save(self) -> None:
71          """Persist the index and metadata to `data/index/`."""
72          import faiss
73
74          config.ensure_dirs()
75          faiss.write_index(self._index, str(config.FAISS_INDEX_FILE))
76          self._meta.to_parquet(config.CHUNK_META_FILE, index=False)
```

Lines 68 to 76 define save(), which persists the store to data/index/. It imports faiss (line 72), ensures the output directories exist (line 74), writes the FAISS index to the configured path (line 75), and writes the metadata frame to parquet without the index column (line 76).

```
77
78      @classmethod
79      def load(cls) -> "VectorStore":
80          """Load a previously-saved index. Fast — no embedding required."""
81          import faiss
82
83          index = faiss.read_index(str(config.FAISS_INDEX_FILE))
84          meta = pd.read_parquet(config.CHUNK_META_FILE)
85          return cls(index, meta)
```

Lines 77 to 85 define load(), the classmethod inverse of save — and fast, because no embedding is needed. It reads the FAISS index from disk (line 83), reads the metadata parquet (line 84), and returns a reconstructed VectorStore (line 85). This is what the notebooks and ask.py call to reuse an already-built index.

```
86
87      # ---- search ----------------------------------------------------------- #
88      @property
89      def size(self) -> int:
90          """Number of chunks in the index."""
91          return int(self._index.ntotal)
```

Lines 86 to 91 define the size property. Marked with @property so it is accessed like an attribute (store.size). It returns index.ntotal — the number of vectors FAISS holds — cast to a plain int (line 91).

```
 92
 93      def search(self, query: str, k: int = config.TOP_K) -> list[Retrieved]:
 94          """Return the `k` chunks most similar to `query`, best first.
 95
 96          Embed the query, ask FAISS for the top-k by inner product, then map each
 97          returned row back to its chunk metadata. FAISS returns -1 for empty
 98          slots when k exceeds the index size, which we skip.
 99          """
100          q = embed_query(query)
101          scores, idxs = self._index.search(q, k)
102          hits: list[Retrieved] = []
103          for score, row in zip(scores[0], idxs[0]):
104              if row < 0:
105                  continue
106              m = self._meta.iloc[int(row)]
107              hits.append(
108                  Retrieved(
109                      chunk_id=m["chunk_id"],
110                      arxiv_id=m["arxiv_id"],
111                      title=m["title"],
112                      text=m["text"],
113                      score=float(score),
114                  )
115              )
116          return hits
```

Lines 92 to 116 define search(), the heart of the class. It embeds the query (line 100), asks FAISS for the top-k by inner product, receiving parallel arrays of scores and row indices (line 101), and prepares an empty hits list (line 102). The loop (lines 103 to 115) pairs each score with its row; it skips rows equal to minus one (line 104 to 105), which FAISS returns as empty slots when k exceeds the index size; otherwise it looks up that row in the metadata (line 106) and appends a Retrieved built from the chunk's fields and the float score (lines 107 to 115). Line 116 returns the hits, already ordered best-first by FAISS.